WeatherFlow — Pipeline Architecture & Interview Mastery

Complete Technical Guide, 15 Interview Q&As; & Verified Resume Bullets for Entry-Level Data Engineers

Role Target: Graduate / Entry-Level Data Engineer	**Stack:** Python, SQL, PostgreSQL, SQLite, Pandas, PyArrow, Pytest
Pipeline Type: Batch Medallion ETL (Bronze/Silver/Gold)	**Testing:** 24/24 Automated Pytest Suite Passing (100%)

1. Verified Resume Bullet Points (Copy-Paste Ready)

These bullets include only genuine, verified features implemented in the codebase without invented claims:

Comprehensive Project Entry (Recommended for Portfolio):

- **WeatherFlow — Automated Weather Data Engineering Pipeline** | *Python, SQL, PostgreSQL, SQLite, SQLAlchemy, Pandas, PyArrow, Pytest, Power BI*
- Engineered an end-to-end Medallion ETL pipeline in Python to extract, validate, clean, and model multi-station meteorological data from NOAA REST APIs and CSV sources.
- Designed an analytical Star Schema in PostgreSQL & SQLite featuring a central fact table (weather_observations), conformed dimensions (dim_date, dim_location), and automated audit tables (pipeline_runs, data_quality_results).
- Built an automated data quality engine that enforces schema contracts and physical domain boundaries (-60°C to +60°C, non-negative rain, min <= max temp), quarantining corrupted rows with automated failure classifications.
- Implemented an idempotent data ingestion pattern using deterministic surrogate hashing (MD5) and SQL ON CONFLICT DO UPDATE (UPSERT) operations, guaranteeing zero duplicate entries on pipeline re-runs.
- Optimized storage and analytical queries by persisting intermediate data in Apache Parquet (PyArrow columnar format) and exporting curated Gold layer summary tables formatted for Microsoft Power BI.
- Authored a comprehensive 24-test suite using Pytest covering unit validation, transformation logic, schema migration, and end-to-end pipeline integration with 100% test pass rate.

2. End-to-End Pipeline Working (The 10 Steps)

Step 1: Ingestion & Extraction	Reads raw multi-station weather records from CSV or the official NOAA CDO v2 API endpoint.
Step 2: Bronze Layer Archival	Creates an immutable, timestamped copy in data/raw/raw_weather_TIMESTAMP.csv for complete data lineage and replayability.
Step 3: Schema Contract Validation	Verifies all 9 mandatory schema columns exist before downstream execution to prevent unexpected KeyErrors.
Step 4: Boundary Validation & Quarantining	Evaluates physical sensor boundaries (-60°C to +60°C, rain >= 0, humidity 0-100%, valid dates). Quarantines corrupted records into data/processed/quarantined_rejected_records.csv with documented error reasons.
Step 5: Type Casting & Normalization	Normalizes dates to ISO YYYY-MM-DD and casts numeric readings to float types safely.
Step 6: Feature Engineering	Derives meteorological seasons (Winter, Spring, Summer, Fall), precipitation intensity categories (None, Light, Moderate, Heavy), and temperature range (T_max - T_min).
Step 7: Silver Layer Persistence	Persists transformed data into weather_cleaned.parquet (columnar, compressed) and CSV for easy inspection.
Step 8: Star Schema Modeling	Extracts conformed dimensions (dim_date, dim_location) and hashes business keys into deterministic MD5 observation IDs.
Step 9: Idempotent Database Load	Loads records using SQL ON CONFLICT DO UPDATE (UPSERT). Re-executing the pipeline safely updates existing records without creating duplicates.
Step 10: Gold Aggregations & Audit Reporting	Executes 7 analytical SQL queries, exports 4 Power BI-ready datasets to data/gold/, and records run metrics in pipeline_runs and data_quality_results.

3. 15 Technical Interview Questions & Answers

Tailored specifically for Entry-Level and Graduate Data Engineer interviews. Practice speaking these answers aloud:

Q1: Can you walk me through the architecture of your WeatherFlow project?

WeatherFlow is an end-to-end weather ETL pipeline built on the Medallion Architecture. In the Bronze Layer, raw observations from local CSVs or the NOAA REST API are extracted and archived with UTC timestamps for immutable lineage. In the Silver Layer, an automated validation engine tests schema contracts and physical domain limits (-60°C to +60°C, non-negative rain). Corrupt rows are quarantined with error reasons, and clean records are enriched with features (seasons, precipitation bins) and persisted in Apache Parquet. In the Gold Layer, clean records are idempotently loaded into a PostgreSQL/SQLite Star Schema, and analytical queries export business-ready datasets for Power BI.

Q2: What is 'idempotency' in data engineering, and how did you implement it in WeatherFlow?

An idempotent pipeline produces the exact same final state regardless of how many times it runs on the same input data. This is critical when schedulers like Airflow retry failed network tasks. In WeatherFlow, I achieved idempotency via two mechanisms: (1) Deterministic Primary Keys: I hashed station_id and observation_date using MD5 to produce immutable surrogate keys. (2) SQL UPSERT: In database.py, the insertion uses 'ON CONFLICT (station_id, observation_date) DO UPDATE SET ...' for facts and 'DO NOTHING' for dimensions. In test_pipeline_rerun_idempotence, executing the pipeline twice against the database maintained the exact same 83 rows.

Q3: Why did you choose a Star Schema over a 3NF normalized schema?

I chose a Star Schema (weather_observations fact table surrounded by dim_date and dim_location) because this is an OLAP analytical workload. Normalized (3NF) schemas are optimized for OLTP write-heavy applications to minimize row updates. Star schemas de-normalize descriptive attributes into conformed dimensions, dramatically reducing the number of JOIN operations required for aggregations (e.g. monthly averages) and mapping directly into BI tools like Power BI without complex data remodeling.

Q4: What data quality checks did you implement, and how do you handle bad data?

I built a three-tier quality framework in src/validation.py: (1) Schema contract check verifying all 9 expected columns exist; (2) Completeness checks ensuring station_id and dates are non-null; (3) Domain boundary checks enforcing physical temperature bounds (-60°C to +60°C), min_temp <= max_temp, precipitation >= 0 mm, and humidity between 0% and 100%. Rather than silently discarding corrupt records, I implemented a Quarantining Pattern: invalid records are saved into quarantined_rejected_records.csv with error tags, and check metrics are recorded in data_quality_results.

Q5: Why did you store intermediate data in Apache Parquet instead of only CSV?

Apache Parquet is an open-source columnar storage format designed for analytics. Unlike CSV, which stores data row-by-row as plain text, Parquet: (1) Organizes data column-by-column, allowing analytical engines to read only requested columns (e.g. temp_avg_c) rather than parsing entire rows; (2) Preserves strong data types (integers, floats, dates) without type-inference overhead; (3) Uses dictionary encoding and Snappy compression, significantly shrinking disk footprint and network I/O.

Q6: Why did you use SQLAlchemy instead of raw database drivers like psycopg2?

SQLAlchemy provides database engine abstraction, connection pooling, and transaction safety. It allowed me to implement dual-engine support: WeatherFlow runs out-of-the-box locally with SQLite (sqlite:///data/weatherflow.db) and seamlessly connects to production PostgreSQL by simply updating the DATABASE_URL environment variable without modifying Python code. Furthermore, using 'with engine.begin() as conn:' guarantees atomic transactions: if a load fails halfway, the transaction rolls back cleanly.

Q7: How did you handle missing values in sensor readings?

In sensor telemetry, missing readings are inevitable. In src/transformation.py, I used domain-specific imputation: if average temperature was missing but min and max were present, I imputed temp_avg_c as (min + max) / 2. Missing weather conditions defaulted to 'Unknown', and precipitation classifications flagged missing readings explicitly as 'Missing'. All evaluations are logged to the data_quality_results table so data consumers understand the data lineage.

Q8: How did you test your data pipeline?

I built a 24-test suite using Pytest covering: (1) Ingestion tests verifying CSV loading, FileNotFoundError handling, and raw file archival; (2) Validation tests asserting boundary rejections on extreme temperatures (999°C), negative precipitation, inverted min/max, and duplicate detection; (3) Transformation tests validating season logic and deterministic hashing; (4) Database tests verifying Star Schema creation, audit logging, and idempotent re-runs; (5) End-to-end integration tests running the full pipeline on a temporary database.

Q9: What is the difference between an OLTP database and an OLAP data warehouse?

OLTP (Online Transaction Processing) systems are designed for day-to-day transactional operations (e-commerce orders, banking), optimized for fast row-level INSERT, UPDATE, and DELETE queries with normalized schemas to avoid redundancy. OLAP (Online Analytical Processing) systems are designed for high-throughput reporting and aggregation over millions or billions of rows (SUM, AVG, GROUP BY). They favor de-normalized Star Schemas, columnar storage, and partitioned architectures.

Q10: How do you prevent sensitive credentials from leaking to GitHub?

I followed the twelve-factor app methodology using python-dotenv. Database credentials (DATABASE_URL) and API tokens (NOAA_TOKEN) are loaded from a local .env file. The .env file is explicitly listed in .gitignore so it is never committed to Git. I committed a sanitized .env.example template containing documentation and placeholders so other developers know how to configure their local environment.

Q11: What would you change if the dataset scaled from thousands to 100 million rows?

If the dataset grew to 100 million daily records: (1) Processing: Migrate transformations from Pandas to PySpark or Polars to distribute operations across a cluster; (2) Storage: Replace local disk storage with cloud object storage (Azure Data Lake Gen2 or AWS S3) partitioned by /year=YYYY/month=MM/; (3) Warehouse: Load the Star Schema into Snowflake, BigQuery, or Amazon Redshift using bulk COPY/MERGE commands rather than iterative UPSERTs; (4) Streaming: Stream high-frequency sensor readings through Apache Kafka into Spark Streaming.

Q12: How would you orchestrate this pipeline in production?

I would orchestrate the pipeline using Apache Airflow: (1) Define a DAG scheduled to trigger daily after NOAA publishes daily observation logs; (2) Break pipeline steps into discrete Airflow Tasks (extract_bronze, validate_quarantine, transform_silver, load_star_schema, export_gold_marts); (3) Implement Task dependencies, retry policies (e.g. 3 retries with exponential backoff on API calls), and Slack/email alerts on failure.

Q13: Explain an analytical SQL query you wrote in analytics.sql.

In Query 2 (Monthly Total Precipitation), I wrote an analytical query aggregating rainfall and rainy days across stations: It JOINs weather_observations with dim_date and dim_location, groups by year, month, and station_name, and calculates SUM(precipitation_mm), AVG(precipitation_mm), and max single-day rain. It uses conditional aggregation: SUM(CASE WHEN w.precipitation_mm > 0.0 THEN 1 ELSE 0 END) to calculate rainy days without needing expensive subqueries.

Q14: What metrics do you track in pipeline_runs and why?

In the pipeline_runs table, I record run_id, pipeline_name, started_at, completed_at, status (SUCCESS/FAILED), records_read, records_clean, records_rejected, records_loaded, and error_message. This operational metadata allows data engineers to track SLA compliance, monitor pipeline run durations over time, detect unexpected drops in source record volume, and diagnose failures quickly from logged error traces.

Q15: What was the biggest technical challenge in this project and how did you resolve it?

The biggest challenge was handling dirty sensor telemetry while maintaining strict pipeline idempotency. Real weather stations periodically send duplicate timestamps, impossible spikes (999°C), and negative precipitation. Hard-failing crashed the pipeline, while silently ignoring them produced inaccurate analytics. I resolved this by decoupling validation into an automated quarantine layer that tags rejection reasons and records audit metrics in data_quality_results. By hashing natural business keys into deterministic surrogate IDs and utilizing SQL ON CONFLICT DO UPDATE, the pipeline became completely idempotent and crash-resilient.

4. Real Analytical SQL Results & Power BI Integration

These are actual SQL outputs produced by WeatherFlow's analytical engine against the database:

Annual Climate Trends by Station (Query 3):

Station Name	Annual Avg (°C)	Record Low (°C)	Record High (°C)	Total Rain (mm)	Observations
MIAMI INTL AP FL US	26.67	18.0	35.5	140.50	12
SEATTLE TACOMA INTL AP WA US	13.13	2.0	29.0	103.00	12
NEW YORK CENTRAL PARK NY US	12.26	-8.5	36.5	145.60	34
CHICAGO O HARE INTL AP IL US	11.22	-17.0	36.0	123.50	25

Seasonal Climate Profile Comparison (Query 7):

Season	Avg Temp (°C)	Lowest (°C)	Highest (°C)	Total Rain (mm)	Daily Rain (mm)	Observations
Winter	3.60	-17.0	27.5	104.30	3.86	27
Spring	15.68	0.0	32.0	111.90	5.89	19
Summer	26.95	14.0	36.5	180.40	9.49	19
Fall	14.89	-2.0	33.0	116.00	6.44	18

Power BI Gold Layer Exports Available in data/gold/:

`powerbi_weather_model.csv`	Full denormalized fact model (observation_date, station, temp, rain, wind, seasons).
`monthly_weather_summary.csv`	Monthly aggregates of temperatures, rainfall, and observation days.
`station_performance_metrics.csv`	Station coordinates, elevations, and 100.0% data completeness scores.
`seasonal_analysis.csv`	Seasonal temperature and precipitation comparisons.

5. Future Enhancements Roadmap (Stages 2–5)

Stage 2: Docker & PySpark	Containerize the application with Docker and migrate transformations to PySpark for distributed compute.
Stage 3: Apache Airflow	Schedule daily batches using an Airflow DAG with task dependency management and automated retries.
Stage 4: Azure Cloud Integration	Store Bronze/Silver/Gold tiers in Azure Data Lake Storage Gen2 with Key Vault secret injection.
Stage 5: Interactive Power BI Dashboard	Connect Power BI Desktop to the Gold CSVs to build interactive slicers, KPI cards, and trend line charts.